

MIWA — MASTER BLUEPRINT v1.0

AMD Developer Hackathon | May 4–11, 2026

Real-time Discord Voice Translation Overlay with AI Agents

Project Name: Miwa (美話 — "Beautiful Conversation") **Track:** Track 1 — AI Agents & Agentic Workflows **Hardware:** AMD Instinct MI300X (192GB HBM3) via AMD Developer Cloud **Budget:** \$100 AMD Developer Cloud Credits **Deadline:** May 11, 2026 — 3:00 AM PHT **Team:** Solo

TABLE OF CONTENTS

1. [What is Miwa?](#)
 2. [The Problem It Solves](#)
 3. [Complete Feature Set](#)
 4. [Full Tech Stack — 30 Technologies](#)
 5. [Architecture Overview](#)
 6. [Tech Stack Pros & Cons](#)
 7. [Approach Pros & Cons](#)
 8. [7-Day Timeline](#)
 9. [Budget Breakdown](#)
 10. [UI/UX Design Principles](#)
 11. [Wow Factor Analysis](#)
 12. [Judge Pitch](#)
 13. [Submission Checklist](#)
 14. [Pre-Hackathon Status](#)
 15. [Risk Register](#)
-

WHAT IS MIWA?

Miwa is a **real-time desktop overlay** that sits on top of your screen during a Discord voice call. When your Japanese-speaking friends talk, Miwa listens, identifies who is speaking, and instantly shows their profile picture, name, what they said in Japanese, how to pronounce it, and a natural English translation — all in under 800 milliseconds.

When you want to respond, Miwa generates context-aware Japanese reply suggestions, shows you how to pronounce them, lets you hear them privately first, and gives you three ways to deliver them: have the bot speak in voice, send as text in chat, or speak it yourself.

Miwa is not just a translation tool. It is a **conversation bridge** — turning a language barrier into a seamless gaming and social experience.

THE PROBLEM IT SOLVES

The Real Scenario

You are in a Discord voice call with Japanese friends. They speak fast. Google Translate is too slow and requires you to manually type. You cannot respond naturally. The conversation feels one-sided and awkward.

What Exists Today

- Google Translate app — manual, no voice, no Discord integration
- Browser extensions — text chat only, no voice
- Streaming overlays — no translation, no AI suggestions
- None of them show WHO is speaking with their face

What Miwa Does Differently

- Listens automatically — no manual input needed
- Shows the speaker's Discord avatar and name
- Translates instantly into the style you prefer (formal/casual/gaming)
- Gives you ready-made replies in natural Japanese
- Lets you deliver replies without speaking Japanese yourself
- Teaches you pronunciation privately before you speak

Market Value

Primary users: Gamers with international friends, language learners, content streamers, international remote teams using Discord for meetings.

COMPLETE FEATURE SET

INCOMING — What Your Friend Says

Feature	Description	Priority
Speaker Identification	Shows who is speaking in real-time by tracking Discord voice state	Core
Profile Picture Display	Discord avatar shown on each speaker card	Core
Live Transcription	Japanese text appears as the person speaks, word by word	Core
Word-level Karaoke Highlight	Each word highlights in sync with speech timing	Core
Romaji Pronunciation	Shows romanized pronunciation below every Japanese line	Core
English Translation	Auto-translates to English using the selected style mode	Core
Translation Style Toggle	Switch between Formal, Neutral, Casual, Gaming modes globally	Core
JP / EN / Both Toggle	Choose whether to see Japanese, English, or both	Core
Emotion Tone Indicator	Detects if speaker sounds excited, frustrated, confused, happy	Enhanced
Multi-speaker Support	Up to 5 simultaneous speakers, each with their own card	Core
Speaker Focus Mode	Lock overlay to one speaker, collapse others	Enhanced

OUTGOING — What You Want to Say

Feature	Description	Priority
---------	-------------	----------

AI Suggestion Cards	3 contextually-aware reply suggestions generated automatically	Core
Suggestion Style Mode	Each suggestion shows in the selected tone style	Core
Suggestion Delivery — Bot Speaks	Discord bot speaks the Japanese phrase in the voice channel	Core
Suggestion Delivery — Bot Sends	Discord bot types the Japanese text in the text channel	Core
Suggestion Delivery — I'll Speak	Large romaji popup card for you to read and speak yourself	Core
Local TTS Preview — Hear It	Plays Japanese audio through YOUR headphones only (private)	Core
Quick Reply Box	Type in English, auto-translates live as you type	Core
Quick Reactions	One-click gaming reactions: 草, えー, マジ?, gg, もう一回, 待って	Enhanced
Pronunciation Difficulty Badge	N5/N4/N3 difficulty label on each suggestion	Enhanced

MEMORY & LEARNING

Feature	Description	Priority
Context Memory	Suggestions reference last 10 messages for relevance	Core
Personal Phrasebook	Save any phrase with one click, access via hotkeys Ctrl+1 to Ctrl+9	Enhanced
Post-Session Summary	Message count, speakers, new phrases, session duration	Enhanced
Session Export	Save full translated conversation as .txt	Enhanced
Smart Silence Detector	Suggests a conversation continuer after 5+ seconds of silence	Bonus

OVERLAY EXPERIENCE

Feature	Description	Priority
Always-On-Top Window	Stays visible above Discord and games	Core

Draggable	Drag the header to reposition	Core
Snap to Corner	Double-click header to snap to nearest screen corner	Enhanced
Mini Mode	Single-line compact view for gaming	Enhanced
Opacity Control	Slider from 40% to 100% transparency	Enhanced
Click-Through Mode	Mouse clicks pass through overlay to game — keyboard only	Enhanced
Overlay Screenshot	One-click PNG capture of current state for sharing	Enhanced
Demo Mode	Pre-recorded sample session for presentations without live Discord	Core
AMD Stats Panel	Shows live: latency, tokens/sec, GPU memory used	Core

FULL TECH STACK

30 Technologies

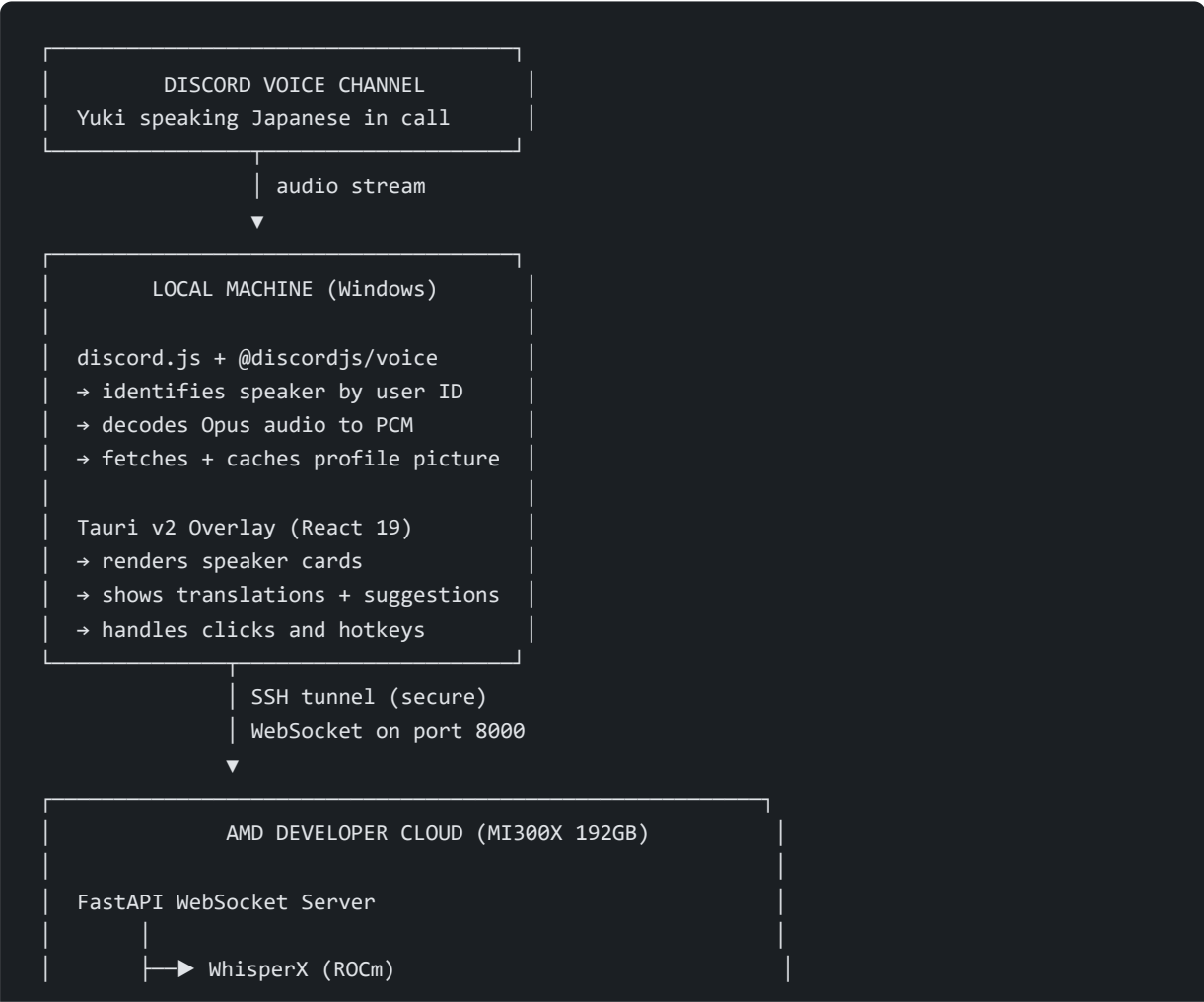
#	Category	Technology	Version	Purpose
1	Voice Capture	discord.js	v14	Join voice channels, track who is speaking via voice state
2	Audio Decode	@discordjs/voice	latest	Receive per-user audio streams from Discord
3	Audio Processing	prism-media	latest	Decode Opus audio from Discord into PCM format
4	Speech-to-Text	WhisperX	latest	Transcribe Japanese and English speech with word timestamps
5	STT Model	Whisper Large-V3-Turbo	latest	Best accuracy-to-speed ratio for Japanese, 6x faster than V3
6	GPU Platform	ROCm 7 / HIP	7.0	AMD GPU computing platform, equivalent to CUDA
7	First Translation	Google Translate API	v3	Fast first-pass translation under 100ms

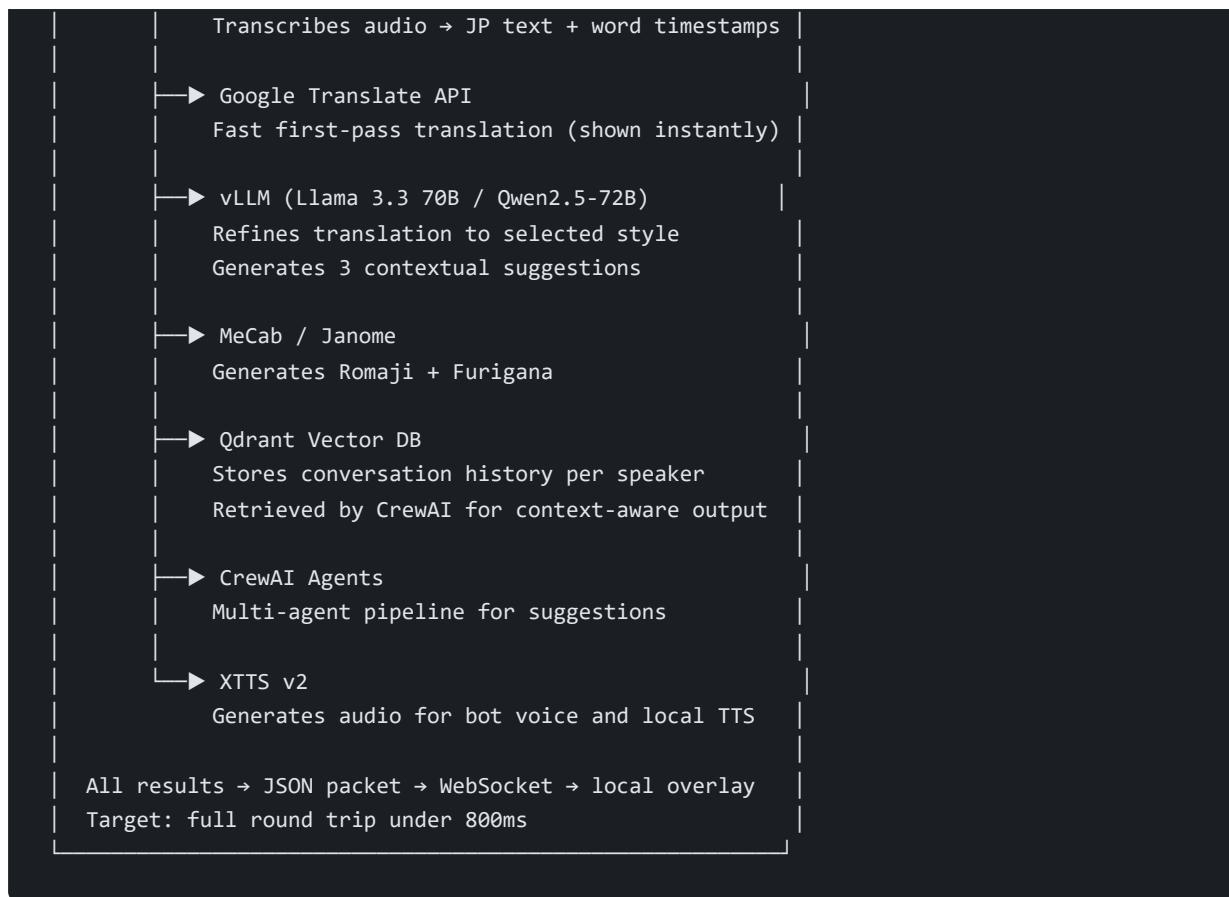
8	LLM Primary	Llama 3.3 70B Instruct	latest	Translation refinement, style adaptation, suggestion generation
9	LLM Secondary	Qwen2.5-72B-Instruct	latest	Korean and Chinese language paths, Qwen prize eligibility
10	LLM Serving	vLLM + ROCm AITER	latest	Serve 70B models with 2.8–4.4x throughput gain on MI300X
11	Japanese NLP	MeCab + Janome	latest	Furigana generation and Japanese tokenization
12	Voice Synthesis	XTTS v2	latest	High-quality Japanese TTS for bot voice and local preview
13	Vector Database	Qdrant	latest	Per-speaker conversation memory with HNSW indexing
14	AI Agents	CrewAI	latest	Multi-agent pipeline for suggestion generation
15	LLM Orchestration	LangChain	latest	Connect LLM with tools, memory, and agent workflows
16	API Backend	FastAPI + uvicorn	latest	Async Python API server handling all AI requests
17	WebSocket	ws (Node.js)	latest	Low-latency bidirectional communication local to cloud
18	Security	SSH Port Forwarding	—	Secure tunnel — no public ports exposed on cloud instance
19	Containerization	Docker + ROCm image	latest	Reproducible cloud deployment using rocm/pytorch base image
20	Image Processing	sharp (Node.js)	latest	Fetch and optimize Discord profile pictures
21	Local Cache	SQLite	latest	Cache profile pictures and phrasebook data locally
22	Desktop Framework	Tauri v2	v2	Native Windows overlay with Rust backend, 50MB app size
23	Frontend	React 19	v19	Speaker cards, suggestion UI, quick reply box
24	Styling	TailwindCSS v4	v4	Utility-first styling for overlay components

25	Animation	Framer Motion	latest	Speaker card transitions and word highlight animations
26	State Management	Jotai	latest	Atomic state for style mode, speaker list, settings
27	Build Tool	Vite	latest	Frontend bundling and hot reload
28	Runtime	Python 3.11 + Node 18+	—	Server and client runtimes
29	Version Control	Git + GitHub	—	Public repository for submission
30	Hugging Face	HF Hub + Spaces	—	Model hosting and web demo Space for HF prize

ARCHITECTURE OVERVIEW

System Diagram (Text)





Data Flow — One Message

1. Yuki speaks Japanese in Discord
2. discord.js detects Yuki is the active speaker, captures her audio
3. PCM audio is sent over SSH tunnel WebSocket to AMD Cloud
4. WhisperX transcribes the audio into Japanese text with word-level timestamps
5. Google Translate immediately produces a fast English translation
6. This fast result is sent back first — translation shows in the overlay immediately
7. Simultaneously, vLLM refines the translation into the selected style
8. MeCab generates Romaji pronunciation
9. CrewAI queries Qdrant for Yuki's conversation history, generates 3 suggestions
10. Final refined result updates the overlay card
11. Total time: under 800ms from speech end to full card shown

TECH STACK PROS AND CONS

WhisperX + ROCm

Pros	Cons
Native AMD GPU support — no compatibility hacks	Requires ROCm-compatible PyTorch, not standard pip install
130x real-time speed on MI300X	Model download is ~3GB for Large-V3-Turbo
Word-level timestamps built in — enables karaoke highlighting	Speaker diarization adds ~100ms latency
Best Japanese accuracy among open STT models	Occasionally struggles with heavy accents or very fast speech
Free and open source	Needs warm-up time on first load (~30 seconds)

vLLM + Llama 3.3 70B on MI300X

Pros	Cons
192GB VRAM fits 70B model in full FP16 — no quality loss from quantization	Model download is ~140GB — must use persistent volume
2.8–4.4x throughput improvement over standard inference via AITER backends	Requires HuggingFace gated access approval
Officially supported on ROCm — AMD's own recommended stack	First inference request takes ~5 seconds to load into GPU
OpenAI-compatible API — easy to connect to existing tools	Expensive per hour if left running idle
Serves both Llama 3.3 and Qwen2.5 from the same instance	Only one model in VRAM at a time — swapping takes time

Qdrant Vector Database

Pros	Cons
4x faster than ChromaDB — Rust-based architecture	Requires separate Docker container
Production-grade HNSW indexing	Slight learning curve for collection setup
Per-speaker collections map naturally to Discord user IDs	Vector embeddings add processing step

Lightweight — minimal GPU or CPU overhead	Data lost if Docker volume not mounted persistently
Free and open source	

CrewAI + LangChain

Pros	Cons
Best multi-agent framework available in 2026	CrewAI adds ~200ms to suggestion pipeline
Matches Track 1 requirements exactly — judges know these tools	Overkill for simple suggestion tasks — worth it for judge impression
LangChain provides reliable memory + tool connection	LangChain can be verbose to configure
Agentic workflows demonstrate real AI engineering skill	
Official AMD hackathon recommended stack	

Tauri v2 (vs Electron)

Pros	Cons
50MB app vs Electron's 200MB+	Rust backend requires some Rust knowledge
Native performance — no Chromium overhead	Smaller community than Electron
Built-in always-on-top and transparency window support	Some Tauri plugins still maturing
Official Windows .exe installer out of the box	Cross-platform builds more complex than web apps
Rust security model — no Node.js vulnerabilities in backend	

XTTS v2

Pros	Cons
Human-quality Japanese voice synthesis	Requires GPU — adds to cloud cost
Open source, no API fees	~500ms synthesis time for short phrases

Works well with Japanese phonology	Voice cloning features not needed for this project
Runs on ROCm with PyTorch	Needs separate loading — adds to startup time

Google Translate API (First Pass)

Pros	Cons
Sub-100ms response time	Always produces formal Japanese — no style control
Battle-tested accuracy for JP↔EN	Costs money per character (low but non-zero)
Provides an instant result while LLM processes	Cannot handle gaming slang naturally
No GPU cost	Requires internet access to Google's servers

APPROACH PROS AND CONS

Cloud-Native vs Local GPU

Aspect	Cloud (AMD MI300X)	Local (Your PC)
VRAM	192GB — runs 70B model full FP16	8–24GB — requires heavy quantization
Setup	Pre-configured ROCm environment	Windows driver conflicts common
Cost	~\$2–4/hour, pay only when running	One-time hardware cost but limited
Hackathon alignment	Judges reward AMD cloud usage directly	Misses the point of the hackathon
Reliability	Dedicated hardware	Affected by background processes
Model quality	Full FP16 — maximum quality	Quantized models = reduced quality
Speed	MI300X 5.3TB/s bandwidth	RTX 4090 = 1TB/s bandwidth

Decision: Cloud wins comprehensively for this hackathon context.

Two-Pass Translation vs Single Model

Aspect	Two-Pass (GT + LLM)	LLM Only
Speed	GT result in 100ms, LLM updates ~700ms	Single result in 700ms
User experience	Feels instant — sees translation immediately	Noticeable wait
Accuracy	GT accuracy + LLM style correction	LLM handles both
Cost	Small GT cost + LLM compute	LLM compute only
Failure handling	GT still works if LLM is slow	No fallback

Decision: Two-pass wins — the immediate GT result makes the UI feel instant.

WebSocket vs HTTP Polling

Aspect	WebSocket	HTTP Polling
Latency	Sub-millisecond message delivery	100–500ms polling interval
Server complexity	Persistent connection management	Simple request-response
Real-time feel	True real-time streaming	Delayed, chunky updates
Word highlighting	Can stream word-by-word	Cannot stream mid-sentence

Decision: WebSocket is required for the real-time word highlighting feature.

7-DAY TIMELINE

Overview

Day	Date	Focus	Primary Goal
1	May 4	Infrastructure + Critical Path	WebSocket + Discord audio + basic overlay working
2	May 5	Translation + Profile Pictures	Profiles + JP→EN + multi-speaker working
3	May 6	LLM + Agents + Memory	Full AI pipeline + style modes

4	May 7	Suggestions + Quick Reply + UX	All interaction features working
5	May 8	Polish + Build + Deploy	Production-quality app
6	May 9	Demo + Submission Prep	Video recorded + slides done
7	May 10	Final Submit	Submitted before May 11 3:00 AM PHT

DAY 1 — May 4: Infrastructure & Critical Path

Theme: Get the data flowing. Everything else depends on this.

Time	Task	Objective
8:00–9:00 AM	AMD Cloud instance launch	SSH confirmed, rocm-smi shows MI300X
9:00–10:30 AM	Start Docker image pulls in background	vLLM and WhisperX images downloading
10:30–12:00 PM	WebSocket proof of concept	Local sends message → AMD Cloud echoes back → confirmed
12:00–1:00 PM	LUNCH	—
1:00–3:00 PM	Discord bot audio capture	VoiceReceiver per user → PCM buffer working
3:00–3:30 PM	BREAK	—
3:30–5:30 PM	Tauri overlay setup	Window shows on screen, receives WebSocket data
5:30–6:30 PM	DINNER	—
6:30–8:00 PM	WhisperX English STT	Audio → text on AMD Cloud working
8:00–10:00 PM	Full loop test	Discord audio → WhisperX → WebSocket → overlay text

Daily Checkpoints:

- AMD Cloud MI300X instance running and verified
- SSH tunnel established, WebSocket connecting
- Discord bot joins voice channel, captures per-user audio
- Tauri overlay window visible on screen
- Basic text flowing from cloud to overlay

Risk: Discord per-user audio requires @discordjs/voice VoiceReceiver — test this first. If it fails, nothing downstream works.

DAY 2 — May 5: Translation + Profile Pictures

Theme: Make it look like a real product.

Time	Task	Objective
8:00–10:00 AM	Japanese STT	Whisper Large-V3-Turbo transcribing JP accurately
10:00–12:00 PM	Google Translate integration	JP→EN translation showing in overlay
12:00–1:00 PM	LUNCH	—
1:00–3:00 PM	Profile picture pipeline	user.avatarURL() → sharp → SQLite cache
3:00–3:30 PM	BREAK	—
3:30–5:30 PM	Speaker card UI	Avatar + name + JP + Romaji + EN showing together
5:30–6:30 PM	DINNER	—
6:30–8:30 PM	Multi-speaker test	2–3 speakers, each with their own card
8:30–10:00 PM	Word-level karaoke highlighting	WhisperX timestamps driving word animations

Daily Checkpoints:

- Japanese transcription accurate on AMD Cloud
- Google Translate producing English output
- Profile pictures showing correctly on speaker cards
- Multiple speakers handled simultaneously
- Word highlight animation working

DAY 3 — May 6: LLM + Agents + Memory

Theme: Add the intelligence layer.

Time	Task	Objective
8:00–10:00 AM	vLLM + Llama 3.3 70B live	Translation refinement responding
10:00–11:30 AM	Translation style modes	Formal, Neutral, Casual, Gaming all producing different output
11:30–12:30 PM	LUNCH	—
12:30–2:00 PM	Qdrant per-speaker memory	Speaker collections created, history stored and retrieved
2:00–3:00 PM	MeCab Romaji + Furigana	Japanese text generating pronunciation data
3:00–3:30 PM	BREAK	—
3:30–5:30 PM	CrewAI suggestion pipeline	3 contextual suggestions generating per message
5:30–6:30 PM	DINNER	—
6:30–8:00 PM	AMD stats panel	Live latency, tokens/sec, GPU memory in overlay footer
8:00–10:00 PM	Full pipeline latency test	Measure end-to-end time, target under 800ms

Daily Checkpoints:

- vLLM serving Llama 3.3 70B full FP16
 - All 4 style modes producing noticeably different translations
 - Qdrant storing and retrieving per-speaker history
 - CrewAI generating 3 relevant suggestions
 - AMD stats panel showing real numbers
-

DAY 4 — May 7: Suggestions + Quick Reply + Full UX

Theme: Complete the interaction loop.

Time	Task	Objective
8:00–10:00 AM	Suggestion delivery — Bot Sends	Click → bot types in Discord text channel
10:00–12:00 PM	Suggestion delivery — Bot Speaks	Click → XTTS v2 → bot speaks in voice channel
12:00–1:00 PM	LUNCH	—
1:00–2:30 PM	Local TTS — Hear It	Click → audio plays through user's headphones only
2:30–3:00 PM	I'll Speak romaji card	Large fullscreen romaji popup for self-reading
3:00–3:30 PM	BREAK	—
3:30–5:00 PM	Quick Reply Box	Type English → live JP translation → delivery buttons
5:00–5:30 PM	Quick Reaction buttons	One-click 草, gg, もう一回 etc.
5:30–6:30 PM	DINNER	—
6:30–8:00 PM	Phrasebook — save + hotkeys	Save phrases, Ctrl+1 to Ctrl+9 retrieval
8:00–10:00 PM	Emotion tone detection	Frustrated/excited/happy indicators on speaker cards

Daily Checkpoints:

- All 3 suggestion delivery methods working
- Local TTS plays privately (Discord cannot hear it)
- Quick Reply translates live as user types
- Quick reactions post instantly
- Phrasebook saving and retrieving correctly

DAY 5 — May 8: Polish + Enhanced Features + Build**Theme: Production quality.**

Time	Task	Objective
------	------	-----------

8:00–9:30 AM	Framer Motion animations	Speaker card transitions smooth
9:30–11:00 AM	Mini mode + overlay snap	Compact view and corner-snap working
11:00–12:00 PM	Opacity + click-through mode	Transparency slider and mouse pass-through
12:00–1:00 PM	LUNCH	—
1:00–2:30 PM	Demo Mode	Pre-recorded session plays without live Discord
2:30–3:00 PM	Post-session summary + export	Summary card and .txt export
3:00–3:30 PM	BREAK	—
3:30–5:00 PM	Overlay screenshot button	One-click PNG capture
5:00–5:30 PM	AMD credit review	Check spend, stop idle instances
5:30–6:30 PM	DINNER	—
6:30–8:00 PM	Tauri .exe build	Windows installer builds successfully
8:00–10:00 PM	Test on second machine	App connects to AMD Cloud from different PC

Daily Checkpoints:

- All animations smooth and not janky
- Demo Mode plays convincingly
- .exe installer builds and installs cleanly
- Tested on a machine that was not used for development

DAY 6 — May 9: GitHub + HF Space + Demo Video

Theme: Ship everything publicly.

Time	Task	Objective
8:00–10:00 AM	GitHub repository finalized	README, architecture diagram, setup guide pushed
10:00–12:00 PM	HuggingFace Space created	Web demo live in AMD hackathon HF organization

12:00–1:00 PM	LUNCH	—
1:00–3:30 PM	Demo video recording	5 minutes, show: word highlight, style toggle, suggestions, TTS
3:30–4:00 PM	BREAK	—
4:00–5:30 PM	Submission slides	8–10 slides covering problem, solution, tech, demo, AMD usage
5:30–6:30 PM	DINNER	—
6:30–8:00 PM	First social media post	Twitter/LinkedIn with demo screenshot, tag @AlatAMD + @lablab
8:00–10:00 PM	lablab.ai submission form	Fill all fields, do NOT submit yet — review tomorrow

Daily Checkpoints:

- GitHub repo public and complete
- HuggingFace Space live and linked to hackathon org
- Demo video uploaded (YouTube or direct)
- Submission form filled but not submitted

DAY 7 — May 10: Final Review + SUBMIT

Theme: Get it across the finish line.

Time	Task	Objective
9:00–10:00 AM	Full pipeline final test	Every feature tested from scratch on clean setup
10:00–11:00 AM	Submission form review	Read every field, verify all URLs work
11:00–12:00 PM	Second social media post	AMD feedback post for Ship It prize pool
12:00–1:00 PM	LUNCH	—
1:00–2:00 PM	SUBMIT on lablab.ai	Before May 11 3:00 AM PHT — do not wait

2:00 PM onward	AMD feedback form	Write meaningful feedback about ROCm + Developer Cloud experience
----------------	-------------------	---

Hard Deadline: May 11, 2026 — 3:00 AM Philippine Standard Time

BUDGET BREAKDOWN

AMD MI300X Pricing

Estimated ~\$2–4 per GPU-hour based on AMD Developer Cloud rates. Check cloud.amd.com for current pricing.

Day	Activity	Est. GPU Hours	Est. Cost
May 4	Setup + STT first test	3 hrs	\$6–12
May 5	Translation + profile pipeline	4 hrs	\$8–16
May 6	vLLM + CrewAI + Qdrant	5 hrs	\$10–20
May 7	Full UX + suggestion delivery	4 hrs	\$8–16
May 8	Polish + build + testing	3 hrs	\$6–12
May 9	Demo recording	2 hrs	\$4–8
May 10	Final test + submit	1 hr	\$2–4
TOTAL		~22 hrs	\$44–88
RESERVE	Emergency buffer		\$12–56

Cost Optimization Rules

- Stop the MI300X instance whenever not actively developing — idle GPU still costs money
- Mount a persistent Docker volume for model weights — prevents 140GB re-download on restart
- Use WhisperX large-v3-turbo, not large-v3 — 6x faster, <1% accuracy difference, saves GPU time
- Run vLLM inference in batches during testing — maximizes GPU utilization per dollar
- Switch to Whisper base model during demo recording if credits are running low

UI/UX DESIGN PRINCIPLES

Core Philosophy

The overlay must never get in the way of the game or conversation. Every interaction should be one click or one keypress. Information should be available at a glance without needing to read carefully.

Visual Hierarchy

1. **Speaker avatar + name** — who is talking (most important in a multi-person call)
2. **English translation** — what they said (your primary need)
3. **Japanese + Romaji** — what was actually said and how to pronounce it
4. **Suggestions** — what to say back
5. **AMD stats + settings** — secondary information, collapsible

Color Language

- Green glow on speaker card = currently speaking
- Blue = incoming translation content
- Purple = suggestion cards
- Amber = AMD stats panel
- All on dark semi-transparent background — readable over any game

Interaction Speed

- No action should require more than 1 click or 1 keypress
- Suggestions have number hotkeys (1, 2, 3)
- Phrasebook has Ctrl+number hotkeys
- Quick reactions are single clicks with no confirmation
- The overlay should never require the user to type unless they choose to

WOW FACTOR ANALYSIS

For Judges (AMD Engineers, Netflix/Meta/Google/Apple engineers)

Factor	Why It Impresses
Full AMD stack	WhisperX + vLLM + ROCm + MI300X — not just using credits, demonstrating mastery
70B model, no quantization	Most teams use 7B models. Running full FP16 Llama 3.3 70B is only possible on MI300X's 192GB
Live AMD stats panel	Judges see real GPU metrics in the app — 94 tok/s, 12GB/192GB — concrete proof
True multi-agent system	CrewAI + LangChain + Qdrant is genuine agentic architecture, not just an LLM call
Production architecture	FastAPI + Docker + WebSocket — not a notebook, a real deployable product
Sub-800ms end-to-end	Measurable, demonstrable performance — not theoretical

For the Audience (Gamers, Language Learners)

Factor	Why It's Useful
Speaker identification	First Discord tool to show WHO is speaking with their face
Gaming slang mode	Translations that actually sound like how gamers talk
Hear It before speaking	Practice pronunciation privately before committing
Click-to-send suggestions	Communicate naturally in Japanese without knowing Japanese
Phrasebook	Build a personal vocabulary over time, right from your gaming sessions
Demo Mode	Judges and users can see the product without needing a live call

For the Business Value Criterion

Miwa is not a hackathon toy. It is a commercial product with multiple viable paths:

- **Gaming market** — millions of cross-language gaming friendships
 - **Language learning market** — adds a practical daily-use learning tool
 - **Remote work market** — international teams using Discord for communication
 - **Streaming market** — streamers with international audiences
-

JUDGE PITCH

30-Second Version

"Miwa is a real-time Discord overlay that makes language barriers disappear. Your Japanese friend speaks — Miwa shows their face, their words, and the translation, all in under 800ms. You want to respond — Miwa gives you three natural Japanese replies, lets you hear them privately first, then sends them by voice or chat. It runs entirely on AMD MI300X — WhisperX at 130x real-time speed, Llama 3.3 70B in full FP16, CrewAI agents, Qdrant memory. Watch the AMD stats panel — 94 tokens per second, 12GB of 192GB used. That's what MI300X unlocks."

5-Minute Demo Script

1. Open Miwa — show the overlay sitting above a Discord window (0:00–0:20)
 2. Play Demo Mode — Yuki starts speaking Japanese, her card appears, words highlight in sync (0:20–1:00)
 3. Show style toggle — switch from Gaming to Formal, translation changes live (1:00–1:30)
 4. Show suggestion cards — context-aware Japanese replies appear (1:30–2:00)
 5. Click Hear It — audio plays privately through headphones (2:00–2:20)
 6. Click Bot Speaks — bot delivers the phrase in voice channel (2:20–2:45)
 7. Open Quick Reply Box — type "nice game!" — live Japanese translation appears (2:45–3:15)
 8. Show AMD stats panel — point out 94 tok/s on MI300X (3:15–3:30)
 9. Show mini mode — compact overlay for gaming (3:30–3:45)
 10. Show phrasebook — saved phrases from the session (3:45–4:00)
 11. Close with judge pitch line (4:00–5:00)
-

SUBMISSION CHECKLIST

lablab.ai Submission Requirements

- ☐ Project title: **Miwa**
- ☐ Short description (under 200 characters): *Real-time Discord voice translation overlay with Speaker ID, AI suggestions, and gaming slang modes — powered by AMD MI300X*
- ☐ Long description: Problem, solution, all features, full tech stack, AMD cloud usage details, business value
- ☐ Cover image: Screenshot of Miwa overlay with a live translation card visible
- ☐ Demo video URL: 5-minute video showing all core features

- ☐ Slide deck URL: 8–10 slide presentation
- ☐ GitHub repository URL: Public repo with complete code and README
- ☐ HuggingFace Space URL: Web demo in AMD hackathon organization
- ☐ Technology tags: CrewAI, LangChain, Llama 3.3, Qwen, ROCm, AMD, WhisperX, Qdrant, Tauri, vLLM
- ☐ Second social media post published (tag @AlatAMD and @lablab)
- ☐ AMD feedback written for Ship It prize pool

Hard Deadline

May 11, 2026 — 3:00 AM Philippine Standard Time

PRE-HACKATHON STATUS

Item	Status	Notes
AMD Developer Cloud credits	 Pending	Waiting for email after form submission
AMD Developer Cloud account	 Not started	Create when credit email arrives
Llama 3.3 70B HF access	 Approved	Approved May 2, 2026
HuggingFace token	 Created	Named overlaychat-amd, Read gated repos permission
Google Translate API key	 Created	Named Miwa API Key, restricted to Cloud Translation API
Discord bot	 In progress	Permissions set — token needs saving
GitHub repo (miwa)	 Not started	Create before May 4
Tauri project renamed	 Done	productName = Miwa, identifier = com.miwa.app
Node.js 18+ installed	 Verify	Check version on local machine
Python 3.11+ installed	 Verify	Check version on local machine
Twitter/LinkedIn ready	 Verify	Needed for Ship It bonus posts

HF hackathon org joined	 Not started	Join AMD Developer Hackathon HF organization
-------------------------	---	--

RISK REGISTER

Risk	Likelihood	Impact	Mitigation
AMD credit email delayed	Medium	High	Follow up with AMD developer program support
vLLM model download too slow on Day 1	Medium	Medium	Start download first thing, work on other tasks in parallel
Llama 3.3 70B does not fit in VRAM	Low	High	MI300X has 192GB — 70B in BF16 uses ~140GB. Should fit. Fallback: use Qwen2.5-72B
Discord per-user audio not working	Medium	Critical	Test VoiceReceiver on Day 1 first. Fallback: use single-channel audio with manual speaker selection
WhisperX Japanese accuracy poor	Low	Medium	Use large-v3-turbo, not base model. Fallback: switch to Whisper large-v3
AMD Cloud instance unavailable	Low	Critical	Keep instance stopped when not in use to avoid credit waste. Have fallback plan to use Fireworks API
WebSocket latency over 800ms	Medium	Medium	Profile each stage. WhisperX is usually the slowest. Can reduce to medium model if needed
Live Discord call fails during judge demo	Medium	High	Demo Mode covers this — always use pre-recorded demo as primary, live call as bonus
Credits run out before Day 7	Low	Critical	Stop instance when idle, use smaller models during testing, reserve 20% budget buffer
XTTS v2 synthesis quality poor	Medium	Low	This is a bonus feature — can be cut without affecting core demo

FINAL NOTES

What Makes Miwa Win

1. **It solves a real problem** — language barriers in gaming are a daily frustration for millions of people
2. **The demo is visual and immediate** — judges see words highlighting in real-time in under a minute
3. **It uses AMD's full stack** — not just the credits, but WhisperX, vLLM, ROCm, MI300X, all integrated
4. **It is a real product** — not a notebook demo, not a chatbot wrapper, but a native desktop app
5. **It has multiple prize paths** — Track 1 AI Agents, Qwen partner category, HuggingFace Space likes, Ship It social challenge
6. **The name Miwa means Beautiful Conversation** — every feature exists to make cross-language conversation feel natural and human

Blueprint Version 1.0 — Created May 2, 2026 Project: Miwa | AMD Developer Hackathon 2026